

EECS 498 • AASE • FALL 2026

# How LLMs Code (Under the Hood)

LECTURE 02 • SEP 3, 2026

The developer who has internalized  
the mechanics has an enormous  
advantage.

# Today's mechanics



## Tokenization

How the model sees your code.



## Context window

The only resource that matters.



## Hallucination

Why the model makes things up.



## Capability vs Knowledge

Two axes of model improvement.

# Tokenization

WHAT THE MODEL SEES

# Tokens, not words

- The model does not see characters
- The model does not see words
- The model sees **tokens** — chunks it has seen in training

Byte-pair encoding: merge most common pairs, repeat until ~100K vocab.

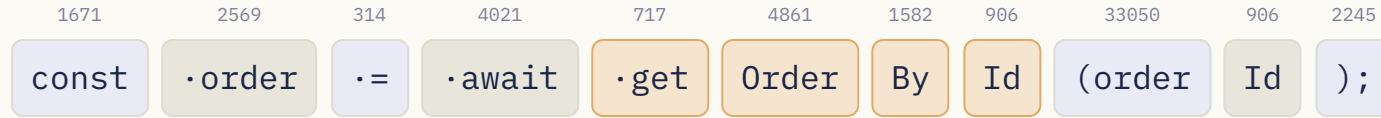
# Where `getOrderById` goes

1. start from characters  
`g e t O r d e r B y I d` 12 tokens
2. merge the commonest adjacent pair. repeat, ~100K times  
`e+r -> er`      `g+et -> get`      `0+rder -> Order`
3. the merges that survived training are the vocabulary  
`get Order By Id` 4 tokens

Nobody chose those four. They are just the pieces that were common enough to earn a slot.

# Real tokens, real IDs

```
const order = await getOrderById(orderId);
```



11 tokens for 42 characters. The four amber ones are exactly what the last slide predicted, and `Id` is token 906 *both* times.

# What tokenization costs you



## Unit of cost

Verbose code costs more — in dollars on a vendor, in RAM and seconds on your own machine.



## Unit of attention

Every token competes for focus. Your identifiers fragment into common pieces, so they carry no special weight.



## 9 tokens

```
def f(x):  
    return x * 2
```

## ~35 tokens

```
def calculate_double_of_input_value(  
    input_value: int  
) -> int:  
    """Return value times two."""  
    return input_value * 2
```

# Context Window

THE ONLY RESOURCE THAT MATTERS

# What the model sees

- **Inside the window:** files, history, repo map, system prompt
- **Outside the window:** *nothing*
- No DB. No web fetch. No retrieval

This is the single most important fact about LLMs for an engineer.

# Three costs of large context



## Cost

Tokens billed, or RAM and seconds on your own box. Attention is quadratic:  $2\times$  context  $\approx 4\times$  work.



## Latency

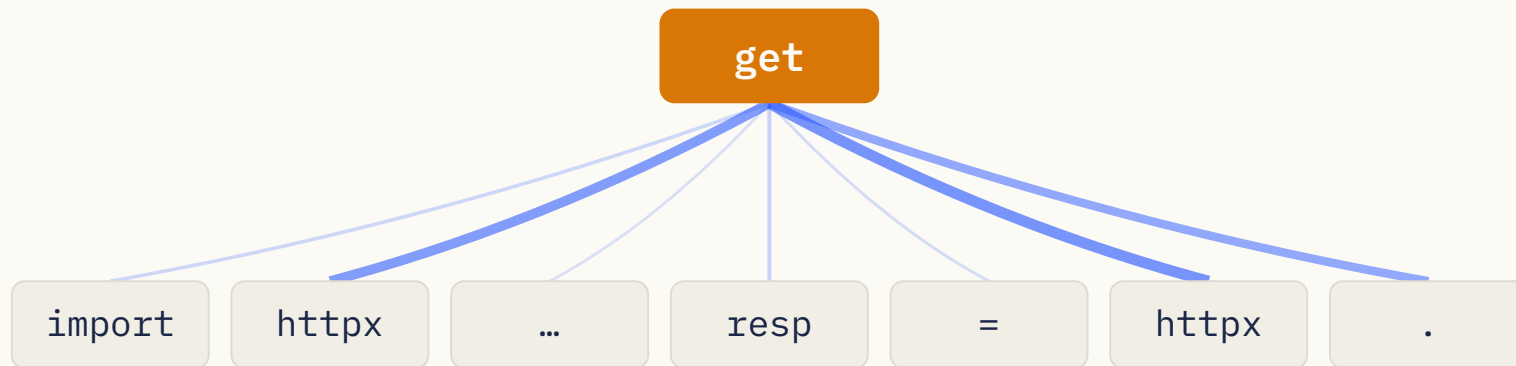
Tens of seconds to first token on a laptop where a tight prompt starts at once.



## Quality

Needle-in-haystack: bigger can be *worse*.

# Weighing the whole window



Thicker line, more weight. It leans on `httpx` and the dot, but it scored *every* token to find that out.

# Every token attends to every other

| Tokens in the window | Pairs scored  | Relative work |
|----------------------|---------------|---------------|
| 1K                   | 1,000,000     | 1×            |
| 2K                   | 4,000,000     | 4×            |
| 8K                   | 64,000,000    | 64×           |
| 32K                  | 1,024,000,000 | 1024×         |

Double the context, quadruple the work. The window is a budget, not a container.

# Aider's friction is the lesson

|                            |                                    |
|----------------------------|------------------------------------|
| <code>/add main.py</code>  | → file enters context              |
| <code>/drop main.py</code> | → file leaves context              |
| <code>/tokens</code>       | → see exactly what's in the window |
| <code>/clear</code>        | → reset history (keep files)       |

Not a limitation. The discipline you'd want anyway.

# Context drift

- History accumulates: every turn you sent, every turn it returned
- Fifty messages in, most of the window is *dead weight*
- The model can't separate what's true now from what was true at message 3

`/clear` between tasks. Claude Code (L05) compacts for you, and compaction is lossy.



CHAT BREAK · 5 MIN

Talk to your neighbor: when a long chat **degrades**, what is happening inside the window?

# Hallucination

THE MISLEADING WORD

# The model is doing what it was trained to do

- Autoregressive sampling
- Pick most likely next token from a distribution
- No fact-check step
- No DB lookup
- No "wait, does this exist?"

# Statistical priors win when context is silent

- Training data: 100× more `import requests` than `import httpx`
- Your project uses `httpx`
- Prompt doesn't mention it
- Model produces `requests` — plausible, wrong for *your* context

# Common hallucinations



## Invented APIs

`user.get_profile()` — plausible, not real.



## Stale knowledge

`pandas 1.x APIs` in a 2.0 codebase.



## Wrong signatures

Wrong kwarg, wrong dtype — subtle.

When the model needs a fact, **show it the fact.**

# What context cannot fix

- "What's the latest version of our internal library?"
- The answer isn't in your repo either, so no `/add` produces it
- The model guesses, *plausibly*, and you cannot tell from the output

Hand it the fact, or build a tool that looks the fact up. You build those in week 5.

# Capability vs Knowledge

TWO AXES



# Capability

**Reasoning skill.** Multi-step, cross-file, holding constraints.

**Scales with:** model size.

**qwen3.5:4b → 9b → frontier.**

# Knowledge

**What the model read.** Recency. Niche depth.

**Scales with:** training data size + freshness.

**Small + recent** can beat large + old.

# Why not the top rung?

④

## qwen3.5:4b

~2.5 GB. Fails fast and fails obviously. Where you start.

⑨

## qwen3.5:9b

~6 GB at Q4. The course default. Part 2 of aider-practice needs it.



## Frontier

Someone else's datacenter. Strong on both axes. You will not run one in this course.

A frontier model covers for you. A 4B *doesn't*. Two weeks from now you'll have run 4b and 9b and felt the gap yourself.

# Match model to task

- **Cost-sensitive batch** (1000× repeats) → smallest model that clears the bar
- **Reasoning-heavy single task** → the most capable model you have
- **Niche library or framework** → capability won't save you. That's a *context* problem
- **Anything you're learning on** → the rung that fails where you erred

Usually right in production, usually wrong in a classroom. The skill is telling which room you're in.

# Diagnostic

WHEN OUTPUT IS WRONG

# Three questions

1. **Context** — did the model have what it needed?
2. **Hallucination** — did it invent an API?
3. **Capability or knowledge** — task too hard? too new?

Tokenization is not on the list. It sits underneath all three and is almost never what you can act on.

Tools wrap the model. The **core inference call** has no lookup.

# What to take away

- **Tokens** are the unit of cost and the unit of attention
- **The context window** is the only resource that matters. Spend it deliberately
- **Hallucination** is pattern completion with no lookup. Fix it with context
- **Capability and knowledge** are different axes. Big against recent

Four mechanics. Every failure you hit this term is one of them wearing a costume.

THIS WEEK

# aider-practice Part 1

- Setup gate and all sixteen lessons due **Fri Sep 11**
- Re-read your own Aider chat logs with today's vocabulary
- Every mechanic from this lecture is *already in them*

Next: L03, the Big Three. Tuesday.



# Questions?

# YouTube Deep Dives



## TOKENIZATION

**Let's build the GPT Tokenizer**

[youtube.com/watch?v=zduSFxRajkE](https://youtube.com/watch?v=zduSFxRajkE)



## ATTENTION

**Attention in transformers, step-by-step**

[youtube.com/watch?v=eMlx5fFNoYc](https://youtube.com/watch?v=eMlx5fFNoYc)

Neither is required. Both go a level below today.